Musanze Harvest Intelligence Service

SWE 3513 Artificial Intelligence - Individual Assignment 2 - Technical Report

Full name	IGIRANEZA Alain Providence
Registration number	25/28075
Institutional username	ug2528075@ines.ac.rw
Verification code	**AI-A2-8075** (seed LLLL = 8075)
Dataset SHA-256	a0ae7a32527e96b1c0f09531b4ebf2806bcb34fa244e1a631d16a8ec853c47df
Commit hash	*insert final commit hash here*

1. Identity and decision problem

Stakeholder. A field officer of a potato cooperative in Musanze who receives farm records in inconsistent spreadsheets and must decide which farms need attention before harvest.

Prediction questions. (1) How many tons will a farm harvest (regression on *yield_tons*)? (2) Is the farm at high production risk, meaning its yield per hectare is below the median (classification on *high_risk*)? (3) Which farms have similar growing conditions (K-means clustering, no targets used)?

Why three tasks. The officer needs a quantity for planning storage and transport, a yes/no flag with a probability for visits and advice, and groups so that one advisory visit can serve similar farms. Training and command-line prediction run locally because internet access may be unreliable; the FastAPI service is for later integration with a mobile application.

Ethical boundary. The system is a classroom decision-support prototype trained on **synthetic** data. It does not replace an agronomist and must never be used to decide access to finance, insurance or public services. Every prediction carries *human_review_required* = *true*. Individual farms must not be ranked or penalised on a model output alone.

How the report is produced. All numbers below are read from *artifacts/*.json* and *test_predictions.csv* by *tools/build_report.py*, so they match the submitted artifacts exactly. The interpretation paragraphs (one regression error, one classification error, one cluster) are my own reading of those results.

2. Data and reproducibility

The dataset is generated by *data/generate_data.py* with *numpy.random.default_rng(8075)*. Base rows = 240 + (8075 mod 41) = 240 + 39 = **279**. Then m = round(0.05 x 279) = **14** values were blanked in each of soil_moisture_pct and fertilizer_kg (independent draws of unique positions) and 3 exact duplicate rows were appended, giving **282** raw rows. The CSV is written with 4 decimals and LF line endings.

Column	Rule	Role
rainfall_mm	Uniform 600-1400	Numeric feature
avg_temp_c	Uniform 12-26	Numeric feature
soil_moisture_pct	Uniform 20-80	Numeric feature
fertilizer_kg	Uniform 0-250	Numeric feature
field_size_ha	Uniform 0.2-4.0	Numeric feature
season	Random A / B	Categorical feature
cooperative_support	Random 0 / 1	Binary feature
yield_tons	0.004 rain + 0.06 soil + 0.02 fert + 1.30 field - 0.25 |temp-18| + 0.70 support + season (A +0.50, B -0.20) + N(0, 0.80), min 0.50	Regression target
high_risk	1 if yield per hectare is below its median	Classification target

Evidence of the deliberate data problems

Check	Result
Raw rows / expected	282 / 282
Missing values	soil_moisture_pct = 14, fertilizer_kg = 14
Exact duplicate rows	3 (removed, keeping the first occurrence)
Rows after cleaning	279
Range / schema / label checks	passed (all five ranges, season in {A,B}, support and high_risk in {0,1}, yield >= 0.50)
Generator matches assignment rule	True
Target-leakage check	passed: no yield_tons / yield_per_ha / high_risk in model inputs

Reproducibility

The SHA-256 of the exact CSV bytes is `a0ae7a32527e96b1c0f09531b4ebf2806bcb34fa244e1a631d16a8ec853c47df` and is stored in *artifacts/data_fingerprint.txt*. Running *run_all.py* twice gave byte-identical CSV, metrics JSON files and test_predictions.csv. *run_all.py* rejects any seed that is not LLLL derived from the student id (for example seed 4671 is refused with a clear message). The same seed is used for the generator, the train/test split, cross-validation folds, every model and K-means.

3. Pipeline design

Split. After removing duplicates (279 rows) a fixed 80/20 split, stratified on high_risk, uses random_state = 8075: 223 training rows and 56 test rows. The test rows are used once, for the final metrics.

Preprocessing (one ColumnTransformer, always inside a Pipeline).

Columns	Steps (fitted on training rows only)
5 numeric features	SimpleImputer(median) then StandardScaler
season	SimpleImputer(most_frequent) then OneHotEncoder(handle_unknown='ignore')
cooperative_support	SimpleImputer(most_frequent), passed through as 0/1

Every saved model is a full Pipeline (preprocessing + estimator), so *predict.py* and *app.py* apply exactly the same transformations as training. Duplicate removal happens before the split (it uses no statistics); imputation, scaling and encoding happen only after it.

Leakage controls. The input list is exactly the seven farm features. yield_tons, yield per hectare and high_risk are never inputs; *check_leakage()* fails the run if any name containing 'yield' or 'high_risk' enters the feature list, and the prediction validators reject such fields. Model selection uses 5-fold cross-validation on the training set only, so the test set does not influence any choice. One honest caveat: the high_risk label itself is defined with the median of the whole generated sample (as the assignment prescribes), which is a property of the data, not of the models.

Candidates.

Task	Baseline	Candidates	Selection rule
Regression	Mean predictor	Ridge, Random Forest, Gradient Boosting	lowest 5-fold CV RMSE (train)
Classification	Majority class	Logistic Regression, Random Forest, Gradient Boosting	highest 5-fold stratified CV F1 (train)
Clustering	-	K-means, K = 2..6	highest silhouette on scaled training data

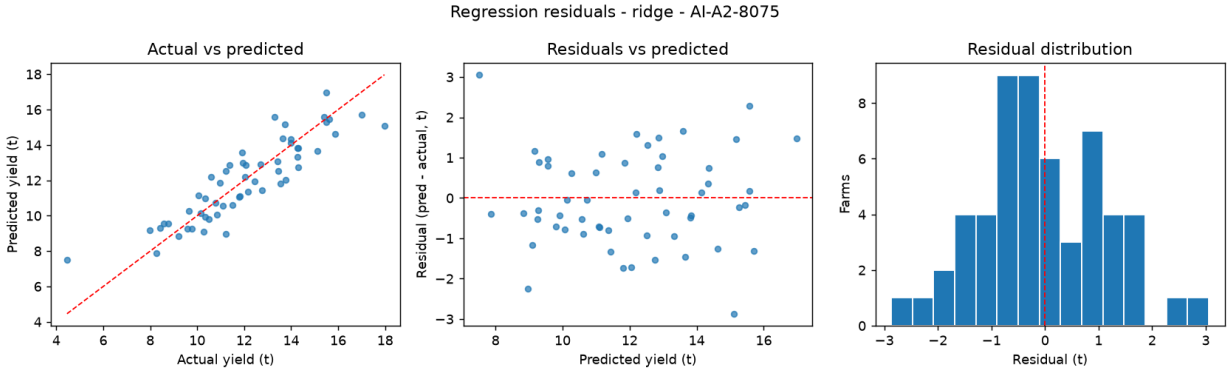
Artifacts. regression_metrics.json, classification_metrics.json, clustering_report.json, test_predictions.csv, run_summary.json, data_fingerprint.txt, three plots, and models/ (regressor, classifier, clusterer, metadata.json). Library versions are recorded in models/metadata.json.

4. Regression and classification

Regression on 56 held-out farms (selected model in bold):

Model	MAE (t)	RMSE (t)	R2	CV RMSE
mean_baseline	2.0293	2.5462	-0.0264	2.5546
ridge	0.9437	1.1539	0.7892	1.0631
random_forest	1.1449	1.5259	0.6314	1.3679
gradient_boosting	1.0168	1.3158	0.7259	1.2178

Selection. ridge has the lowest 5-fold cross-validated RMSE on the training set (1.0631 t) among the candidates; it was chosen before looking at the test set. On the held-out test set it reaches MAE 0.9437, RMSE 1.1539, R2 0.7892 versus the mean baseline's RMSE 2.5462 and R2 -0.0264.



Residual reading. Residuals are centred near zero (mean -0.054 t) and show no strong curve against the predicted value, so the linear model captures the main structure; the mean baseline has R2 near zero, so the features carry real signal. The two tree ensembles scored worse than Ridge here, which fits a dataset whose yield formula is almost linear and small (223 training rows).

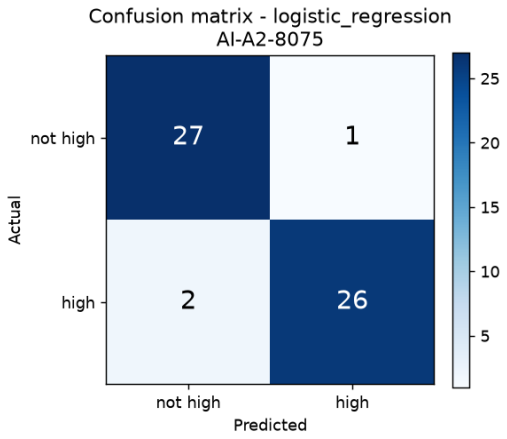
Important regression error (my interpretation). The largest miss is farm row_id 68: actual 4.46 t, predicted 7.51 t (over-prediction of 3.06 t). Its fertilizer_kg is one of the deliberately blanked values, so the pipeline imputed the training median (about 122 kg). Replaying the generator shows the true value was about 23 kg; at 0.02 t per kg that gap alone is about 2.0 t of the 3.06 t error. The farm is also small (0.31 ha) with dry soil (25 %), a corner of the data with few training examples. Lesson: median imputation hides a real difference between farms; a real system should flag records with imputed inputs.

Classification (positive class = high_risk):

Model	Precision	Recall	F1	ROC-AUC	CV F1
majority_baseline	0.0000	0.0000	0.0000	0.5000	0.0000
logistic_regression	0.9630	0.9286	0.9455	0.9898	0.9454
random_forest	0.9286	0.9286	0.9286	0.9732	0.9033
gradient_boosting	0.9259	0.8929	0.9091	0.9579	0.9144

Selection. logistic_regression has the highest 5-fold stratified cross-validated F1 on the training set (0.9454); chosen before looking at the test set. Test result: precision 0.9630, recall 0.9286, F1 0.9455, ROC-AUC 0.9898. The majority baseline cannot find any high-risk farm (recall 0.0000).

4b. Classification interpretation



Confusion matrix. TN 27, FP 1, FN 2, TP 26. The majority baseline predicts every farm as the majority class, so its precision, recall and F1 are 0 (ROC-AUC 0.5) - it never finds a single high-risk farm, which makes accuracy alone misleading. For the field officer a false negative (a high-risk farm missed) costs more than a false positive (an unnecessary visit), so recall deserves attention alongside precision.

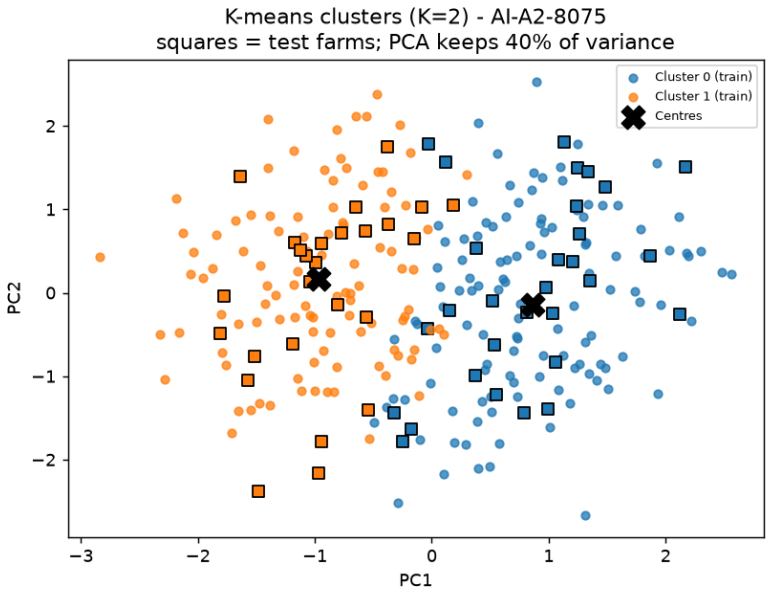
Important classification error (my interpretation). Farm row_id 161 was not a high-risk farm but received probability 0.81. It has a large field (2.79 ha) and heavy rain (1241 mm), and yield per hectare of high_risk is defined relative to the median, so large farms with a good but not extreme yield sit close to the decision boundary where noise (SD 0.80 t) decides the label. The other misclassified test farms have probabilities of 0.38 and 0.48, i.e. close to the 0.5 threshold: borderline cases rather than confident mistakes.

5. Clustering

The seven input features are imputed, scaled and one-hot encoded on the training rows, then K-means (n_init = 10, random_state = 8075) is run for K = 2..6. Targets are not used.

K	Silhouette	Inertia
2	0.1449	1083.3
3	0.1328	968.8
4	0.1414	869.4
5	0.1383	803.0
6	0.1419	751.3

Choice of K. K=2 has the highest silhouette (0.1449), which indicates no substantial structure (Kaufman-Rousseeuw scale). Clusters were built from the seven input features only; yield and risk are shown for description, not used. The silhouette values for all K lie between 0.133 and 0.145, so the choice of K=2 over K=4 or K=6 is a small difference. This is expected: the features were drawn independently and uniformly, so the farms do not form natural groups. The clusters are therefore a convenient split of one continuous cloud, not distinct farm types.



Cluster	Train / test farms	Centre (rain, temp, soil, fert, field)	Support / season A	Mean yield / high-risk rate*
0	118 / 30	882, 19.5, 53.2, 158, 1.54	0.45 / 0.54	11.23 t / 0.28
1	105 / 26	1122, 19.4, 45.1, 85, 2.75	0.55 / 0.44	12.14 t / 0.74

* yield and risk are descriptive only; they were not used to form clusters.

Plain-language profiles

Cluster 0 (118 training farms): smaller fields.

Cluster 1 (105 training farms): higher rainfall, lighter fertilizer use, larger fields.

One farm cluster (my interpretation). Cluster 1 groups larger fields (2.75 ha on average) with higher rainfall but lighter fertilizer use (85 kg against 158 kg). Its high-risk rate is 0.74 versus 0.28 in cluster 0, although its mean yield in tons is higher: large fields produce more tons but, with little fertilizer per farm, less yield per hectare. Operationally, the officer could prioritise fertiliser advice for this group. Because the silhouette is low, this should be treated as a hint to investigate, not as a farm type.

6. API and testing

Endpoint	Behaviour
GET /health	200 {status, models_loaded, verification_code}; 503 if models are missing
GET /model-info	verification code, student, SHA-256, selected models, test metrics, feature ranges, library versions, synthetic-data notice
POST /predict	typed body (pydantic, strict types, ranges, extra fields forbidden); 200 with yield, risk label, probability, cluster, profile, verification code; 422 with field-level messages

Valid request: samples/valid.json returns the yield estimate, risk prediction, probability, cluster, cluster profile and verification code (AI-A2-8075); the command-line output and the API response use the same code path (*src/inference.py*).

Invalid requests. samples/invalid.json combines four faults and the CLI reports each one: missing field_size_ha, rainfall_mm = 2500 outside [600, 1400], avg_temp_c = 'warm' (wrong type), season = 'C'. The API returns HTTP 422 with *{error: validation_failed, details: [{field, message}]}*; the same rules also reject unexpected fields such as yield_tons (leakage).

Automated tests (tests/test_system.py, 11 tests): valid inference; missing field; wrong type; range error; leakage fields; dataset reproducibility and fingerprint; deliberate missing/duplicate evidence; seed rejection; metrics agree with test_predictions.csv; API health/model-info/predict; API 422 for four invalid bodies.

Test summary. ========================= 11 passed, 1 warning in 5.33s =========================

Failure behaviour. Missing model files give a clear instruction to run run_all.py (CLI exit code 3, API 503). Invalid input gives exit code 2 / HTTP 422 and never reaches the models.

7. Model card and reflection

Item	Statement
Intended use	Teaching prototype that shows how a field officer could triage farms and plan harvest quantities.
Data	Fully synthetic, generated from the stated formula; no real farm was measured.
Limitations	Models learn the generator, not real agronomy. Inputs are limited to seven features; no pests, soil type, variety or price. Small sample (279 rows). Clusters are weak. Predictions outside the training ranges are rejected.
Risks	Over-trust in a number; imputed values hidden from the user; synthetic accuracy mistaken for real accuracy; the median-based label always marks about half the farms as high risk.
Prohibited use	Deciding loans, insurance, subsidies or any public service; ranking or punishing farmers; replacing an agronomist.
Human oversight	Every response has human_review_required = true; an agronomist or field officer must review each recommendation.

With real data I would: collect measured yields and field records, report which inputs were imputed, check fairness across districts and farm sizes, calibrate the probabilities, monitor drift each season, and validate the clusters with agronomists before using them.